PROJECT DOCUMENTATION

AI WeatherWise - Intelligent Weather Forecasts and Personalized Weather Insights

1. Introduction

Project Title : AI WeatherWise

Team Leader : SANJAY M

Mail ID :ncas2427sz1041@ncas in

NM ID : 045CA27BA735905BEF8469323343D085

Team ID : 6ab4bed3b5170a9b58b4a152

1.1. Team Members

1. SUBASH R

Mail ID : ncas2427sz1317@ncas.in

NM ID : 28AD34EA2FFC538AF4945B2137DAC0E3

2. VIMALASHWARA P

Mail ID : ncas2427sz1142@ncas.in

NM ID : 1FFBA48E72A5B0F8FA543A7A338AB562

3. SATHISH M

Mail ID : ncas2427sz1143@ncas.in

NM ID : FA8D2309B3B8A27B221477632CEB405D

AI WeatherWise is a RESTful backend platform developed using Node.js and Express.js, with MongoDB and Mongoose for data persistence. The system provides weather information and weather-based insights through secure APIs. It supports JWT-based authentication, password hashing, favorite location management and a fallback mode that keeps the API flow functional when external service access is unavailable.

The backend is designed as a modular, headless service. Routing, authentication, controllers, services and database models are separated to improve maintainability and scalability.

2. Project Overview

AI WeatherWise provides a centralized backend service for retrieving weather information and generating weather-based recommendations. The platform is accessed through REST APIs and can be tested using Thunder Client or Postman.

- User registration and login
- JWT-based authentication
- Favorite location management
- Weather information retrieval

- Weather-based recommendations and AI insights
- MongoDB data persistence using Mongoose
- Request sanitization and centralized error handling
- Fallback support for external-service limitations

3. Scenario-Based Case Study

3.1. Background

A travel enthusiast frequently visits different cities and needs a convenient way to track local weather conditions and receive useful suggestions based on the current weather.

3.2. Problem

- Difficulty tracking multiple favorite weather locations efficiently.
- Lack of personalized recommendations for outdoor activities or clothing.
- Need for natural-language summaries instead of only raw weather metrics.
- Need for secure storage and synchronization of user preferences.

3.3. Solution

AI WeatherWise provides a centralized backend service for weather retrieval and weather-based insight generation. User and location information are stored in MongoDB, while secured routes control access to protected operations.

3.4. Usage

- User Usage: manage favorite cities, fetch weather information and request recommendations.
- Developer/Admin Usage: monitor API health, manage user registrations and verify service integrations.
- Public Weather Usage: consume the available weather endpoint for current metrics.

3.5. Outcome

The system reduces the effort required to interpret weather measurements by combining structured weather data with recommendation-oriented API responses while keeping the backend modular and secure.

4. Software Requirements

- **Operating System:** Windows 10/11, macOS or Linux
- **Node.js:** Version 16 or above
- **npm:** Version 8 or above
- **MongoDB:** MongoDB database with Mongoose ODM
- **AI Integration:** Google Gemini AI integration
- **API Testing:** Thunder Client / Postman
- **Code Editor:** Visual Studio Code or equivalent

5. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent
- **RAM:** 8 GB minimum; 16 GB recommended

- **Storage:** At least 1 GB of available workspace

6. System Architecture

AI WeatherWise follows a modular backend architecture in which API requests move through the Express server gateway, authentication middleware and controllers before reaching the weather/AI services or MongoDB data layer. External weather and AI services are integrated through the service layer.

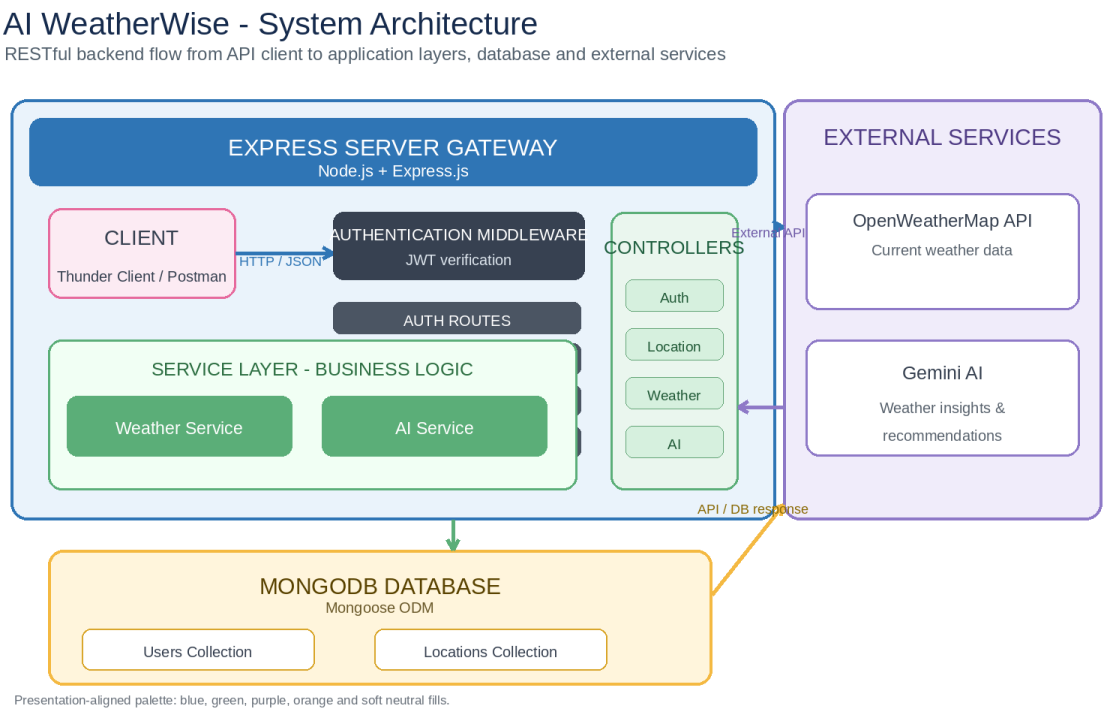


Figure 6.1: System Architecture of AI WeatherWise

6.1. Architecture Components

- **Client / API Interface:** Sends HTTP requests and receives JSON responses through Thunder Client/Postman.
- **Express Server Gateway:** Handles routing, HTTP methods, JSON parsing and middleware execution.
- **Authentication Middleware:** Validates JWT tokens for protected routes.
- **Controller Layer:** Processes requests and coordinates application workflows.
- **Service Layer:** Handles weather retrieval and weather/AI business logic.
- **Model / Database Layer:** Uses Mongoose models to interact with MongoDB collections.
- **External Services:** Provides current weather data and AI insight/recommendation services.

7. Entity-Relationship (ER) Diagram

The database model contains two core entities: User and Location. A registered user can store multiple favorite locations, giving the relationship a one-to-many (1:N) structure.

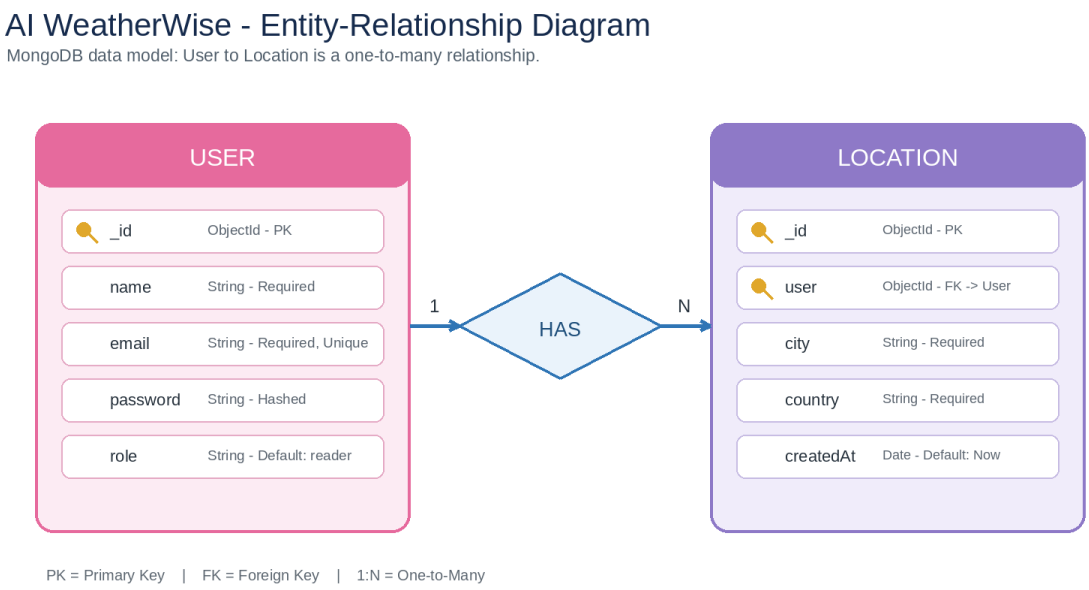


Figure 7.1: Entity-Relationship Diagram of AI WeatherWise

7.1. User Entity

- **_id:** Primary Key - ObjectId
- **name:** String, Required
- **email:** String, Required, Unique
- **password:** String, Hashed, Required
- **role:** String, Default: reader

7.2. Location Entity

- **_id:** Primary Key - ObjectId
- **user:** Foreign Key referencing User
- **city:** String, Required
- **country:** String, Required
- **createdAt:** Date, Default: Now

7.3. Relationship

User (1) - HAS - Location (N): one registered user can save multiple favorite locations.

8. Key Features

- Role-based route guarding using JWT authentication.
- Favorite location CRUD operations.
- Current weather retrieval through the weather API.
- Weather-based recommendation endpoint.
- MongoDB persistence through Mongoose models.
- Sanitization and centralized error handling.

- Fallback operation when external service access is unavailable.

9. Roles and Responsibilities

9.1. Admin

- Manage user accounts.
- Monitor API health and system logs.
- Monitor database integrity.

9.2. Registered User

- Maintain a list of favorite locations.
- Request current weather information.
- Request weather-based insights and recommendations.
- Manage authentication-related information.

10. User Flow

The user flow represents the sequence followed when a user interacts with the WeatherWise backend.

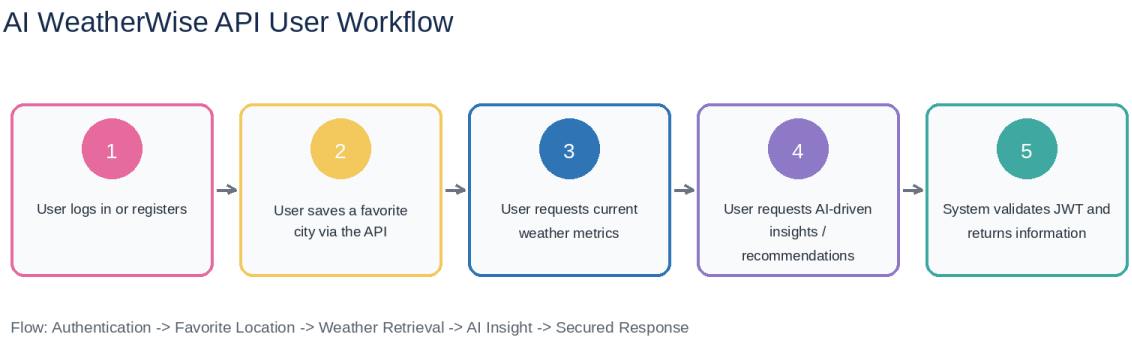


Figure 10.1: AI WeatherWise API User Workflow

11. MVC Architecture

AI WeatherWise follows the Model-View-Controller pattern. Because the project is a headless backend, the View is represented by the API routing layer rather than a conventional HTML user interface.

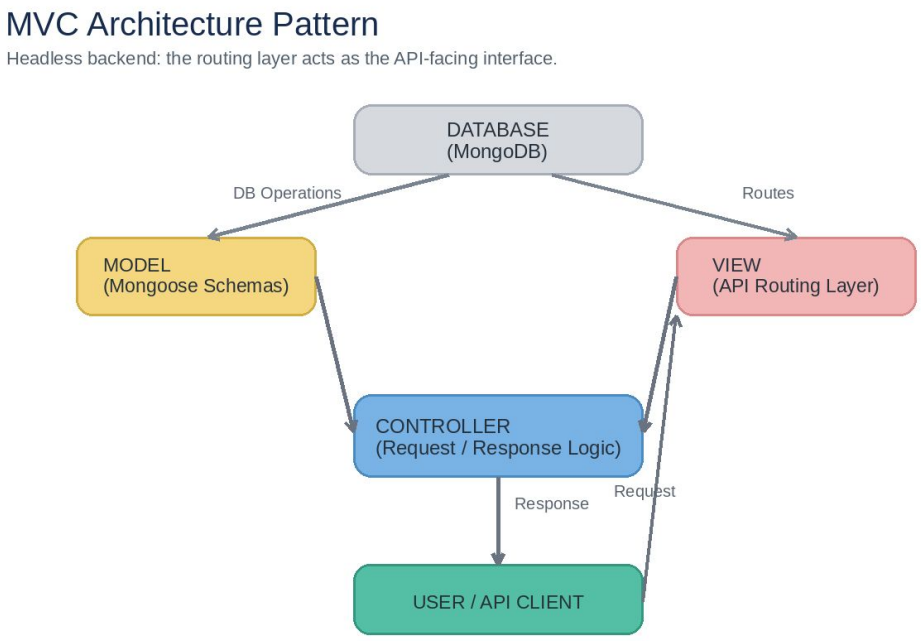


Figure 11.1: MVC Architecture Pattern for AI WeatherWise

11.1. Model Layer

Defines the MongoDB data structures through Mongoose schemas and handles data mapping and validation.

11.2. Controller Layer

Coordinates incoming requests, applies workflow logic, calls the required models/services and prepares JSON responses.

11.3. View / API Routing Layer

Maps HTTP requests to the appropriate controller handlers and exposes the backend through REST endpoints.

12. Project Setup and Configuration

12.1. Project Initialization

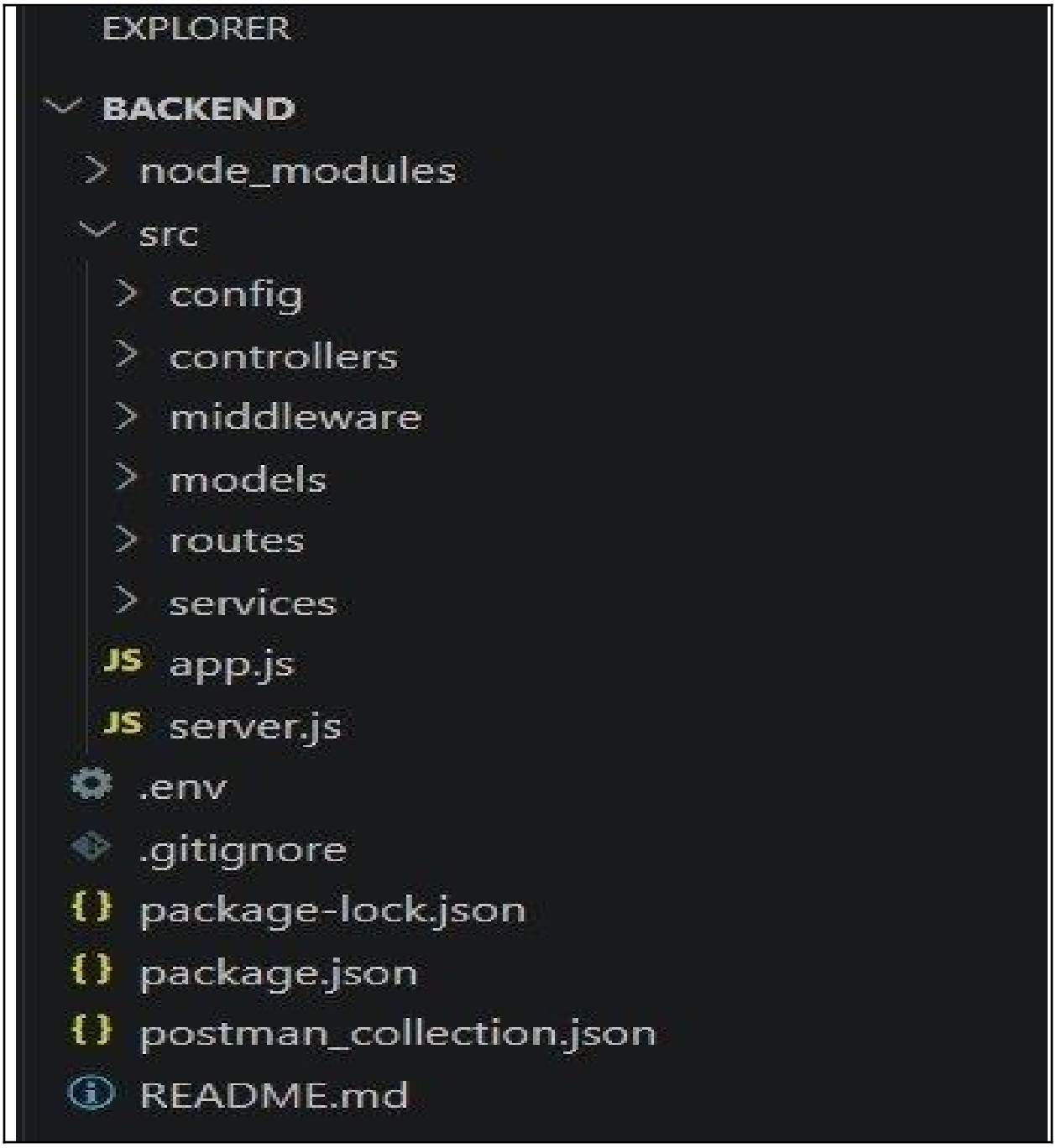
The project is opened in Visual Studio Code and initialized as a Node.js application.

```
npm init -y
npm install
npm run dev
```

12.2. Project Folder Structure

The project separates the main backend responsibilities into configuration, controllers, middleware, models, routes and services. Entry-point files are used to configure and start the application.

Screenshot 01 - Project Structure



12.3. Environment Configuration

Environment variables are used to store runtime configuration and secret values. The actual credentials are kept private and are not included in the public repository.

- **PORT:** Backend server port
- **MONGO_URI:** MongoDB connection string
- **JWT_SECRET:** Secret used for JWT signing
- **GEMINI_API_KEY:** Key for Gemini AI integration

13. Database Configuration

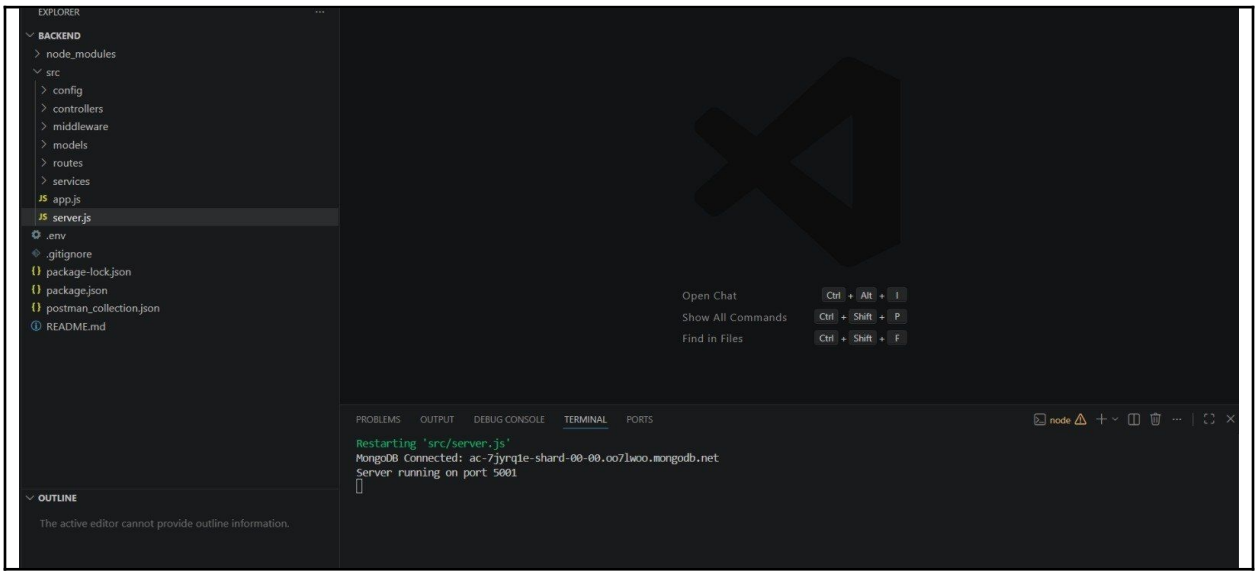
MongoDB is used as the persistent data store and Mongoose is used to define schemas and perform database operations. The User and Location models provide the core application data structure.

The server establishes a successful MongoDB connection before the application begins serving requests.

14. Project Execution

The development server is started from the VS Code terminal. The completed execution state displays a successful MongoDB connection followed by the server listening on port 5001.

Screenshot 02 - Server Running and MongoDB Connected



15. API Testing

The backend APIs were tested using Thunder Client. The following four API tests were prepared for the project demonstration.

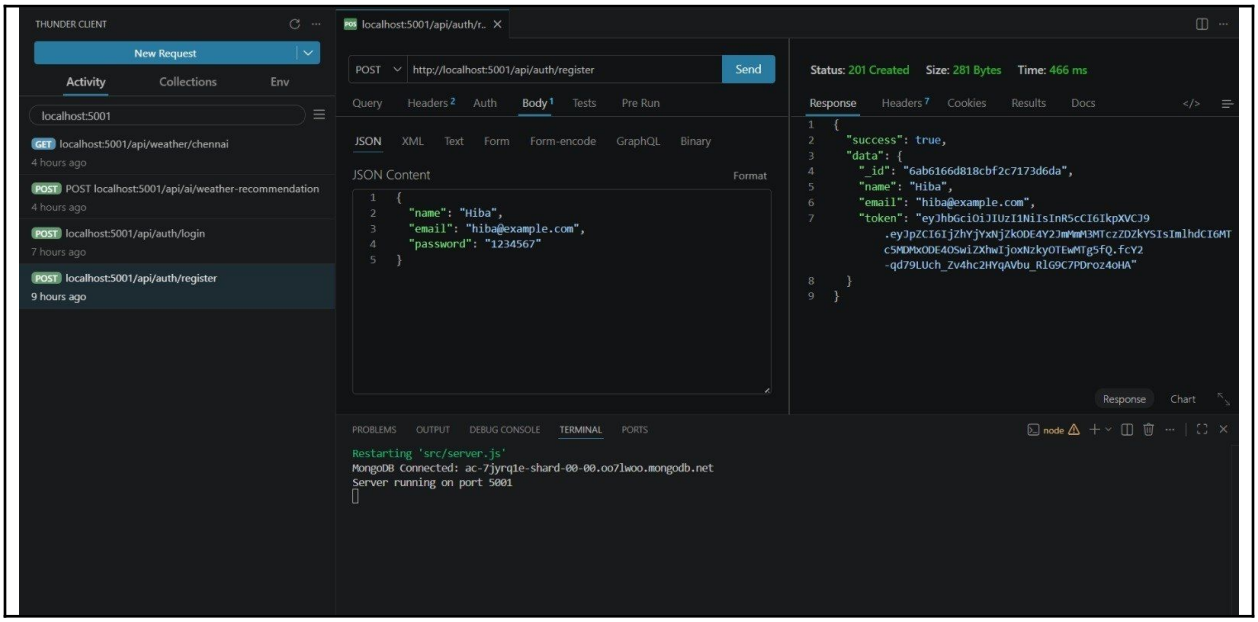
15.1. Register API

Method: POST

Endpoint: http://localhost:5001/api/auth/register

Purpose: Registers a new user account and stores the user record in MongoDB.

Screenshot 03 - Register API Response



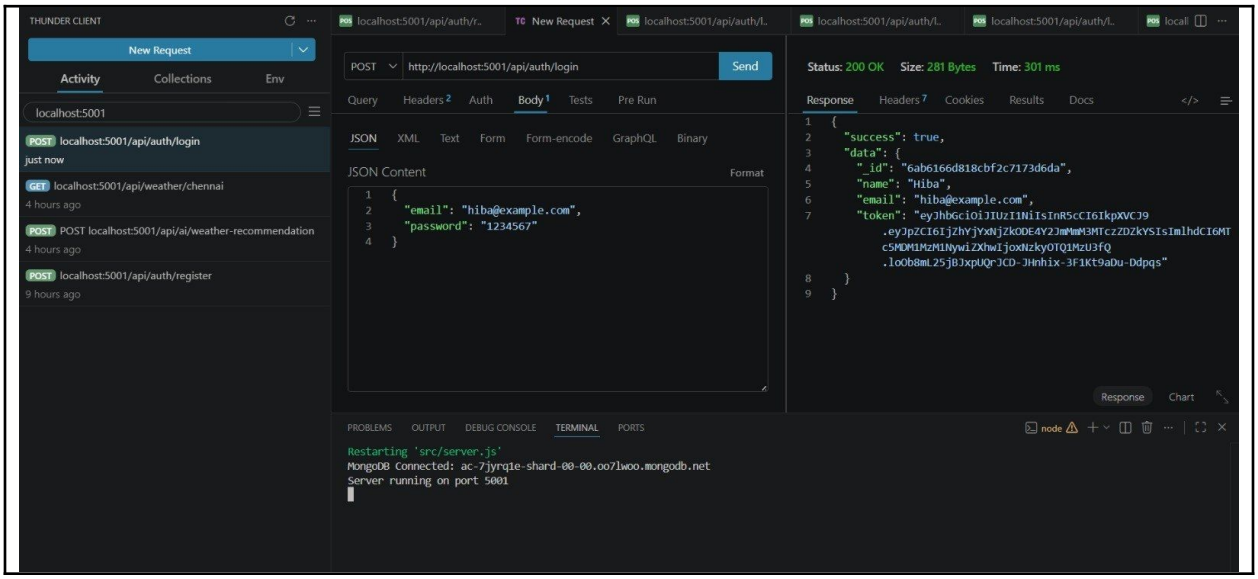
15.2. Login API

Method: POST

Endpoint: http://localhost:5001/api/auth/login

Purpose: Verifies the registered user credentials and returns authentication information, including a JWT.

Screenshot 04 - Login API Response



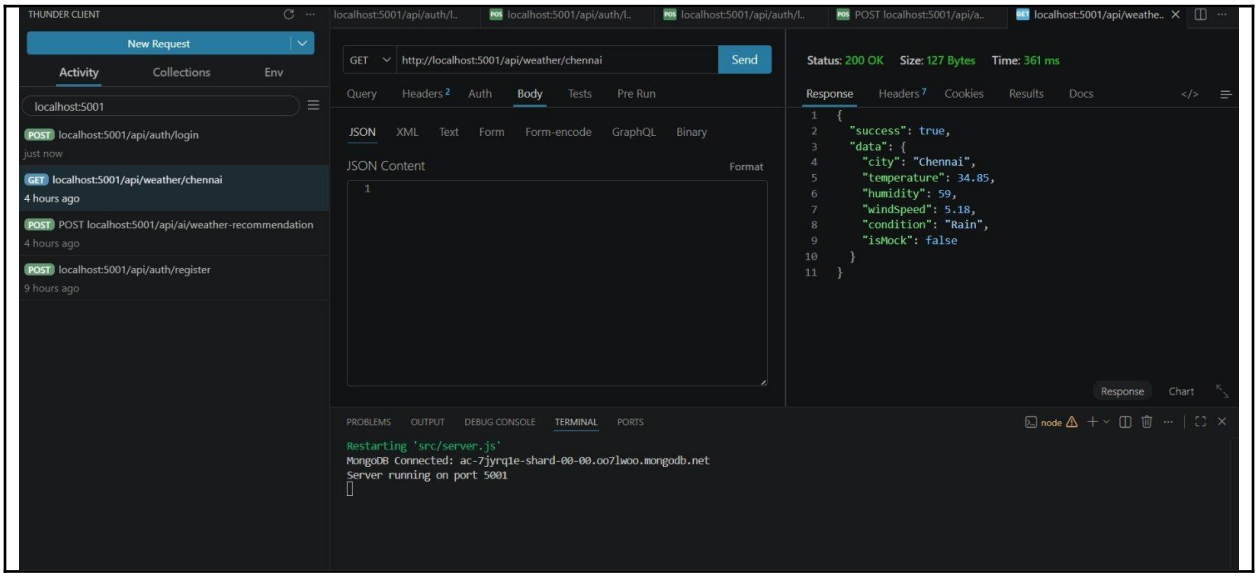
15.3. Weather API - Chennai

Method: GET

Endpoint: http://localhost:5001/api/weather/chennai

Purpose: Retrieves weather metrics for Chennai, including temperature, humidity, wind speed and the current condition.

Screenshot 05 - Weather API: Chennai Response



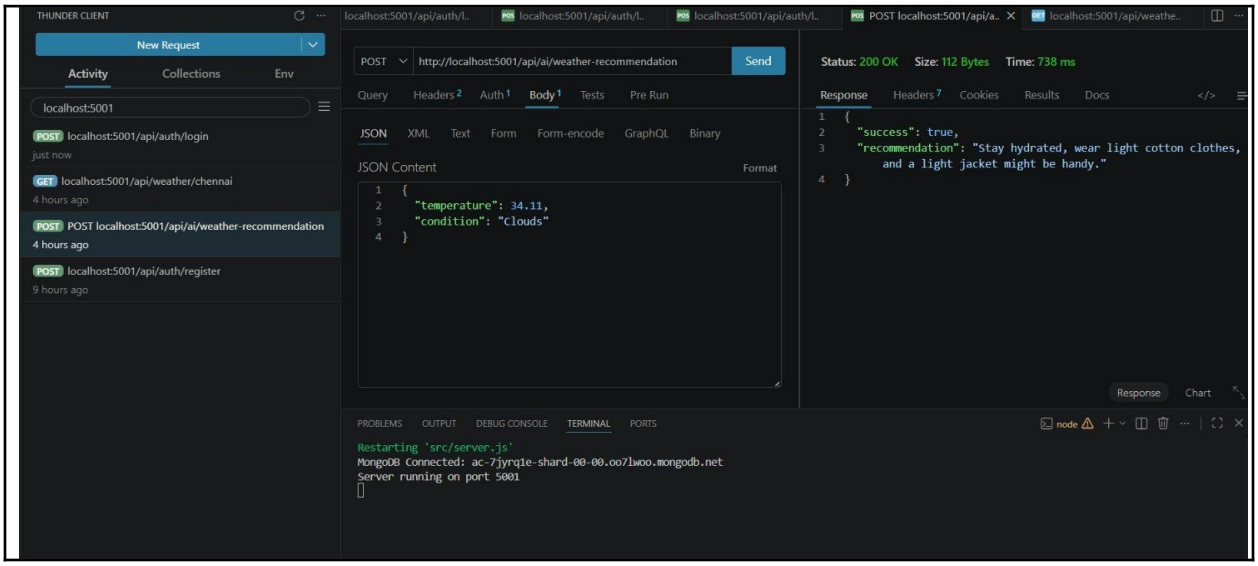
15.4. Recommendation API

Method: POST

Endpoint: http://localhost:5001/api/ai/weather-recommendation

Purpose: Returns a weather-based recommendation through the backend recommendation workflow.

Screenshot 06 - Recommendation API Response



16. Testing Summary

User Registration: Successful - 201 response

Endpoint: /api/auth/register

User Login: Successful - 200 response

Endpoint: /api/auth/login

Weather Retrieval: Successful - 200 response

Endpoint: /api/weather/chennai

Weather Recommendation: Successful - 200 response

Endpoint: /api/ai/weather-recommendation

17. Authentication and Security

- JWT-based authentication is used for protected API requests.
- Passwords are stored using hashing rather than plain text.
- Protected routes validate the authentication token.
- Input sanitization and centralized error handling are included.
- Secret credentials such as database passwords, API keys and JWT values are kept outside the public documentation and repository.

18. Fallback Support

The project includes a fallback mechanism so that the API workflow remains available when external service credentials or access are unavailable. This keeps the backend request-processing and response flow demonstrable under such conditions.

19. Video Demonstration

API Testing Video

Google Drive Link :

https://drive.google.com/file/d/1lLHR8-pPwI_Se5LxNZ-53rwwcI8Sjf9Z/viewusp=drive_link

20. Project Repository and Code

Public GitHub Repository:

https://github.com/sanjayakash357-svg/Vetri-Thiran-Payirrchi-Thittam.git

Code Drive Link : https://drive.google.com/file/d/1oaZF0GMW-kmChICDgUBdNo-LjVTfKZBr/view?usp=drive_link

The repository is intended to contain the project code and the phase-wise documentation required for submission.

21. Known Limitations

- The backend depends on external services for some weather and AI-related operations.
- External API access may be unavailable depending on credentials or service availability.

- The project is designed as a headless backend and does not include a conventional frontend interface.

22. Future Enhancements

- Expand favorite-location management capabilities.
- Add richer personalized weather insights.
- Extend weather-based activity and clothing recommendations.
- Add more external weather-data capabilities.
- Introduce a dedicated frontend interface in a future phase if required.

23. Conclusion

AI WeatherWise is a modular RESTful backend that combines weather retrieval, authentication, database persistence and recommendation-oriented processing. The project demonstrates secure API-based operations using Node.js, Express.js, MongoDB and Mongoose, with an architecture designed for maintainability and extension.